

INTELIGENCIA ARTIFICIAL Y MINI ROBOTS

INTELIGENCIA ARTIFICIAL GENERATIVA

Desarrollo de los ejercicios propuestos en la sección 7.11

Ejercicio 1. Definición de skills propias para un agente de IA

Presentado por:

Edinson Sanchez Fuentes (edsanchezf@unal.edu.co)
Brayan Alejandro Muñoz Pérez (bmunozp@unal.edu.co)
Joan Sebastián Roberto Puerto (jroberto@unal.edu.co)

Docente: José J. Martínez P.
Universidad Nacional de Colombia, Sede Bogotá
Octubre de 2026

Contenido

1. Introducción.....	3
1.1 Metodología.....	3
2. Ejercicio 1. Definición de skills propias para un agente de IA.....	4
2.1 Planteamiento: qué es una skill.....	4
2.2 Las skills de referencia de AI Hero.....	4
2.3 Diseño del conjunto.....	5
2.4 Skill de captura de requerimientos (numeral 4).....	6
2.5 Skill de documentación de tickets (numeral 1).....	7
2.6 Skill de implementación con pruebas (numeral 2).....	7
2.7 Skill de explicación en HTML (numeral 3).....	8
2.8 Estructura del programa.....	8
2.9 Verificación de las skills.....	8
2.10 Demostración: el flujo completo sobre los documentos de la clase.....	9
2.11 Conclusiones.....	13
3. Uso de herramientas de IA Generativa.....	15
3.1 Herramientas empleadas.....	15
3.2 Prompts utilizados.....	15
3.3 Alcance y limitaciones observadas.....	15
4. Programas, repositorio y ejecución.....	17
4.1 Estructura de los archivos.....	17
4.2 Ejecución en Google COLAB.....	17
4.3 Ejecución local con Jupyter.....	17
4.4 Instalación de las skills en un agente.....	17
5. Bibliografía.....	18

1. Introducción

Este documento presenta el desarrollo del ejercicio 1 propuesto en la sección 7.11 del capítulo *Inteligencia Artificial Generativa*. El ejercicio se entrega como un notebook de Jupyter ejecutado, con sus resultados incluidos, acompañado de las cuatro skills que define, y en este documento se describen sus partes, sus resultados y las conclusiones que se derivan de ellos, conforme a los parámetros de entrega del curso.

El capítulo define una skill como un paquete de carpetas de instrucciones, scripts y recursos, generalmente anclado por un archivo Markdown llamado `SKILL.md`. El ejercicio pide definir, con base en las skills publicadas en AI Hero, cuatro skills propias: una para documentar un ticket de trabajo, una de implementación con generación de código y pruebas, una de explicación que genere un HTML del trabajo realizado y una que lea y capture requerimientos de documentos provistos.

1.1 Metodología

Una skill es texto: siempre se puede escribir una que parezca razonable, y sin una forma de comprobarla no es posible distinguir una que funciona de una que solo está bien redactada. Las verificaciones realizadas fueron las siguientes:

- **Partir de una referencia real.** Las skills de AI Hero se tomaron de su repositorio público (`mattpocock/skills`) y cada skill propia indica qué conserva de ellas y qué cambia.
- **Comprobación del formato.** Un programa verifica en las cuatro skills las reglas del formato *Agent Skills* (nombre, descripción, archivos referenciados) y las convenciones propias del ejercicio.
- **Prueba de activación.** Como el agente decide qué skill usar leyendo solo la descripción, se midió si las cuatro descripciones se distinguen entre sí frente a solicitudes típicas.
- **Validadores con casos negativos.** Cada script de validación se probó también con una entrada incorrecta, para confirmar que detecta lo que debe (un ticket mal escrito, una prueba que falla por la razón equivocada).
- **Uso encadenado sobre documentos reales.** Las cuatro skills se aplicaron una tras otra a los siete documentos de la clase, desde la captura de sus requerimientos hasta el informe HTML, con todas las cifras leídas de las ejecuciones. La detección de requerimientos se midió contra una referencia escrita a mano.

2. Ejercicio 1. Definición de skills propias para un agente de IA

Enunciado: con base en los skills en <https://www.aihero.dev/> defina sus propias skills. En principio 4: 1. Un skill para documentar un ticket de trabajo, toda la descripción del trabajo por hacer. 2. Un skill de implementación, generación de código y pruebas. 3. Un skill de explicación, que genere un HTML que explique en detalle el trabajo realizado. 4. Un skill que lea y capture requerimientos de documentos provistos.

2.1 Planteamiento: qué es una skill

Una skill le enseña a un agente de IA (Claude Code, Codex, Cursor, Copilot y otros que adoptaron el formato abierto *Agent Skills*) un procedimiento que de otro modo habría que explicarle en cada conversación. Es una carpeta con un archivo `SKILL.md` obligatorio, que empieza con un encabezado YAML con los campos `name` y `description`, y opcionalmente plantillas, scripts y otros recursos.

El agente no carga todo al mismo tiempo. La carga es progresiva, en tres niveles, y de ella se derivan las reglas de diseño que se aplicaron en todo el ejercicio:

Tabla 1. Niveles de carga de una skill y su consecuencia para el diseño.

Nivel	Qué se carga	Cuándo	Consecuencia para el diseño
1	name y description	Siempre, de todas las skills instaladas	La descripción decide si la skill se usa: debe decir qué hace y cuándo usarla
2	Cuerpo de <code>SKILL.md</code>	Cuando la descripción coincide con la solicitud	Instrucciones cortas; los detalles van en otros archivos
3	Plantillas, referencias y scripts	Solo si las instrucciones los mencionan y hacen falta	Los scripts se ejecutan sin leerse: su código no ocupa contexto

De la tabla sale el principio central del diseño: lo que necesita **criterio** (decidir si una frase es un requerimiento, redactar un criterio de aceptación) va como instrucción en `SKILL.md`; lo que es **mecánico** y debe salir igual siempre (extraer el texto de un PDF, validar un ticket, ejecutar una prueba, armar el HTML) va como script.

2.2 Las skills de referencia de AI Hero

AI Hero (Matt Pocock) publica un conjunto de skills de ingeniería de software organizadas como un flujo de trabajo, con licencia MIT. Las que sirven de base al ejercicio son las siguientes:

Tabla 2. Skills de AI Hero usadas como referencia.

Skill	Qué hace
<code>/grill-with-docs</code>	Entrevista exhaustiva al usuario para afinar un plan; documenta glosario y decisiones
<code>/to-spec</code>	Convierte la conversación en una especificación: problema, solución, historias de usuario, decisiones y pruebas
<code>/to-tickets</code>	Parte la especificación en tickets como rebanadas verticales, cada uno con lo que lo bloquea y criterios de aceptación
<code>/implement</code>	Implementa un ticket usando <code>/tdd</code> , ejecuta las pruebas y hace commit
<code>/tdd</code>	Ciclo rojo → verde: pruebas en las costuras acordadas, comportamiento por la interfaz pública, sin pruebas tautológicas

Skill	Qué hace
/code-review	Revisión del trabajo terminado
/teach	Enseña un tema con lecciones en HTML autocontenidas, cuidadas e imprimibles

Cada una de las cuatro skills propias toma ideas de estas y las adapta al enunciado:

Tabla 3. Correspondencia entre las skills propias y las de AI Hero.

Skill propia (numeral)	Se basa en	Qué se conserva	Qué cambia
capturar-requerimientos (4)	/grill-with-docs, /to-spec	Registrar lo ambiguo como pregunta en lugar de inventar	La fuente son documentos, no una entrevista; cada requerimiento lleva cita y línea; scripts de extracción y detección
documentar-ticket (1)	/to-tickets, /to-spec	Rebanadas verticales, bloqueado por, criterios de aceptación, sin rutas ni código	Plantilla completa (alcance, riesgos, estimación, definición de terminado); criterios Dado/Cuando/Entonces con identificador; validador
implementar-con-pruebas (2)	/implement, /tdd, /code-review	Costuras, rojo antes de verde, una rebanada a la vez, qué es una buena prueba	Script que ejecuta cada fase y deja bitácora; script de trazabilidad criterio → prueba; lista de revisión
explicar-en-html (3)	/teach	HTML autocontenido, legible e imprimible	Explica trabajo terminado con evidencia real; secciones obligatorias; generador con plantilla

Las skills de AI Hero dependen de una configuración previa (/setup-matt-pocock-skills) que define el gestor de tickets del proyecto. Las propias no la necesitan: trabajan con archivos locales y mencionan GitHub como opción.

2.3 Diseño del conjunto

Las cuatro skills forman una cadena: la salida de cada una es la entrada de la siguiente, y los identificadores conectan todo, de modo que cualquier fila del HTML final se puede rastrear hasta una línea del documento original.

Flujo de las cuatro skills y cadena de identificadores

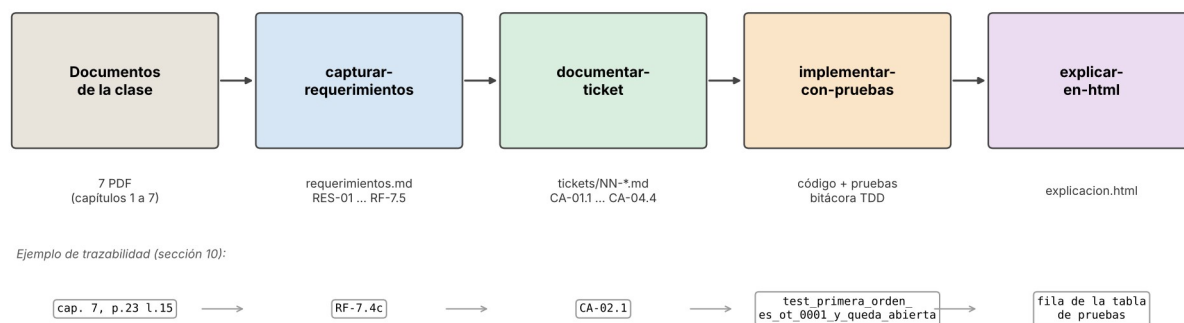


Figura 1. Flujo de las cuatro skills y ejemplo de la cadena de identificadores.

Tabla 4. Convenciones comunes a las cuatro skills.

Convención	Razón
Nombre en minúsculas con guiones, en español, verbo + objeto	Es el formato exigido para name y deja claro qué hace la skill
Descripción = qué hace + «Úsala cuando...» con las palabras que diría el usuario	Es lo único que el agente ve antes de decidir (nivel 1)
SKILL.md de menos de 100 líneas, con enlaces a plantillas y scripts	Carga progresiva: los detalles solo entran al contexto si hacen falta
Scripts en Python con la biblioteca estándar (salvo pypdf y python-docx para leer esos formatos)	Corren en cualquier máquina donde corra el agente
Cada script termina con código 0 (correcto) o 1 (hay que corregir)	El agente sabe si debe corregir algo sin interpretar el texto

2.4 Skill de captura de requerimientos (numeral 4)

Se describe primero porque es la que arranca el flujo. Un documento mezcla contexto, teoría, necesidades y condiciones; lo que se pide suele estar repartido entre secciones de ejercicios, viñetas, frases con «debe» y notas al final. Si el agente lo resume de memoria, se pierden requerimientos o aparecen otros que nadie pidió. La skill capturar-requerimientos separa el trabajo en dos partes:

- **Parte mecánica.** `extraer_texto.py` saca el texto completo de PDF, DOCX, TXT o Markdown conservando la página. `detectar_requerimientos.py` propone candidatos con las cuatro reglas de la tabla siguiente, y a cada uno le asigna tipo (RF funcional, RNF no funcional, RES restricción), prioridad MoSCoW según el verbo («debe» → Debe, «se sugiere» → Debería, «podría» → Podría; Elegible si hay regla de selección) y un nivel de confianza.
- **Parte de criterio.** El agente revisa cada candidato: descarta el contexto, parte los compuestos, reescribe cada uno como «El sistema debe...», escribe un criterio de verificación y convierte lo ambiguo en preguntas abiertas.

Tabla 5. Reglas del detector de requerimientos.

Regla	Qué detecta	Ejemplo en los documentos del curso
Obligación	Frases con debe, se requiere, se sugiere, podría, shall, must	«Los programas deben presentarse en Jupyter Notebook o en COLAB»
Sección de requerimientos	Ítems entre un título Ejercicios, Problemas, Requerimientos, Actividades, Entregables o Tareas y la siguiente Bibliografía, Referencias, Anexos o Conclusiones	«5.9. Ejercicios» ... «Punto libre.»
Verbo inicial	Ítems y frases que empiezan en infinitivo (Crear, Maximizar) o imperativo (Tome, Simule, Desarrolle)	«Crear un sitio en GitHub para la entrega...»
Regla de selección	escoja, elija, desarrolle tres de los seis, de los últimos cuatro haga tres	«De los ejercicios 1 y 2, escoja 1 para realizar; y hacer 3 o 4.»

La confianza es alta dentro de una sección de requerimientos, media en un ítem de una lista introducida por una frase terminada en «:» y baja en el resto, que casi siempre es prosa; no descarta nada, ordena la revisión. Con PDF reales importaron cuatro detalles: un título de sección es la línea completa y empieza en mayúscula (así, una línea partida que termina en «conclusiones.» no cierra la sección), un ítem incluye sus líneas de continuación y puede cruzar un salto de página, una regla de selección al final de un ítem se reporta aparte, y «podría» se revisa antes que «debe» porque «debería» contiene la raíz *deb-*.

La regla central de la skill es la trazabilidad: cada requerimiento lleva la cita textual, el documento, la página y la línea de donde salió; lo que no tiene fuente es un supuesto y se declara como tal.

2.5 Skill de documentación de tickets (numeral 1)

El enunciado pide documentar «toda la descripción del trabajo por hacer». El criterio adoptado es que un ticket está completo cuando otra persona, o un agente que no vio la conversación, puede hacer el trabajo sin preguntar. De /to-tickets se tomaron tres ideas: cortar el trabajo en **rebanadas verticales** (cada ticket entrega un comportamiento completo y demostrable), declarar qué tickets **bloquean** a cada uno, y describir el qué y el por qué sin rutas de archivos ni código, que se vuelven obsoletos. La única excepción es la **interfaz acordada**: las firmas de las funciones donde se observarán las pruebas, que son el contrato con la skill siguiente.

La plantilla tiene once partes: metadatos (tipo, prioridad, estimación, requerimientos, bloqueado por, estado), contexto, objetivo, alcance con *fuera de alcance* explícito, interfaz acordada, criterios de aceptación, casos borde, dependencias y riesgos, preguntas abiertas y definición de terminado. Los criterios se escriben en formato *Dado / Cuando / Entonces* con un identificador CA-NN.n, que es lo que la skill de implementación pone en cada prueba.

El script `validar_ticket.py` convierte la plantilla en reglas comprobables: secciones presentes y no vacías, metadatos completos, estimación en la serie de Fibonacci (un ticket de más de 8 puntos se debe partir), criterios con formato e identificador únicos, ningún marcador de la plantilla sin llenar, y un aviso cuando un criterio usa palabras que no se pueden verificar (rápido, fácil, correctamente).

2.6 Skill de implementación con pruebas (numeral 2)

Generar código es la parte fácil para un modelo de lenguaje; lo difícil es tener evidencia de que hace lo que pide el ticket. La skill `implementar-con-pruebas` sigue el ciclo de desarrollo guiado por pruebas de /tdd: en la fase **roja** se escribe una prueba para un criterio y se ve fallar; en la **verde** se escribe el mínimo código para que pase. Ver la prueba fallar primero tiene una razón: una prueba que pasa antes de escribir el código no está probando nada. Y debe fallar por la razón correcta; si falla porque el módulo no existe, no se sabe si la aserción sirve. Por eso la skill pide crear primero la interfaz con `raise NotImplementedError`.

Sobre la base de AI Hero se agregaron dos scripts. `ciclo.py` ejecuta una prueba en la fase indicada, comprueba que el resultado sea el esperado (falla en rojo, pasa en verde), distingue las fallas por la razón equivocada (`ImportError`, `SyntaxError`, `NameError`) y registra todo en `bitacora_tdd.json`. `trazabilidad.py` cruza los criterios del ticket con los docstrings de las pruebas y señala los criterios sin prueba. Una lista de revisión (`REVISION.md`) cierra el trabajo.

2.7 Skill de explicación en HTML (numeral 3)

De /teach se tomó la forma (un archivo HTML autocontenido, cuidado tipográficamente e imprimible) y se cambió el propósito: no enseña un tema, rinde cuentas de un trabajo terminado. Hay dos decisiones de diseño. La primera es separar contenido y presentación: el agente escribe `contenido.json` (texto, tablas, fragmentos de código) y `generar_html.py` lo convierte con una plantilla fija; así todos los informes tienen el mismo aspecto, el código siempre se escapa bien y el agente no gasta contexto escribiendo CSS. La segunda es exigir **secciones obligatorias** con evidencia real: resumen, problema, decisiones, funcionamiento, código clave, pruebas, limitaciones y cómo reproducir. El generador avisa si falta una sección o si un fragmento de código pasa de 25 líneas. La plantilla no usa recursos externos, tiene tema claro y oscuro y estilos de impresión.

2.8 Estructura del programa

Tabla 6. Secciones del notebook del Ejercicio 1.

Sección	Contenido
1. Planteamiento	Qué es una skill, su estructura y la carga progresiva en tres niveles.
2. Skills de AI Hero	Skills de referencia y correspondencia con las propias.
3. Diseño del conjunto	Flujo, convenciones y figura de la cadena de identificadores.
4 a 7. Las cuatro skills	Una sección por skill: justificación y celdas <code>%%writefile</code> que escriben <code>SKILL.md</code> , plantillas y scripts en <code>skills/</code> .
8. Verificación del formato	Función <code>verificar_skill</code> y árbol de archivos.
9. Activación	Similitud TF-IDF entre solicitudes y descripciones.
10. Demostración	Los siete documentos de la clase; detección y medición contra una referencia; revisión del agente y <code>requerimientos.md</code> ; estado frente al repositorio; cuatro tickets y su validación; implementación con ciclos rojo-verde; suite, trazabilidad y bitácora; HTML final.
11. Instalación	Dónde se copian las skills y empaquetado en <code>.zip</code> .
12. Conclusiones	Conclusiones, uso de IA generativa y referencias.

Las skills no se escriben a mano fuera del notebook: las celdas `%%writefile` las crean, de modo que el notebook es a la vez su documentación y su fuente. La función auxiliar `correr` ejecuta los scripts de las skills tal como lo haría un agente, como procesos independientes, y devuelve su salida y su código de terminación.

2.9 Verificación de las skills

Formato. Las cuatro skills cumplen todas las reglas comprobadas:

Tabla 7. Verificación del formato de las cuatro skills.

Skill	name válido e igual a la carpeta	Caracteres de description (≤ 1024)	Dice cuándo usarla	Líneas de SKILL.md	Archivos referenciados / rotos	Scripts que compilan
capturar-requerimientos	Sí	395	Sí	62	3 / 0	2 de 2
documentar-ticket	Sí	417	Sí	51	2 / 0	1 de 1

Skill	name válido e igual a la carpeta	Caracteres de description (≤ 1024)	Dice cuándo usarla	Líneas de SKILL. md	Archivos referenciados / rotos	Scripts que compilan
implementar-con-pruebas	Sí	346	Sí	65	3 / 0	2 de 2
explicar-en-html	Sí	380	Sí	49	3 / 0	1 de 1

Activación. Con varias skills instaladas, el agente elige cuál cargar leyendo solo las descripciones. Para medir si se distinguen se representaron las cuatro descripciones y ocho solicitudes típicas con TF-IDF sobre n-gramas de caracteres de 3 a 5 (para que «implementa» e «implementar» compartan rasgos sin lematizar), y cada solicitud se asignó a la skill de mayor similitud coseno.

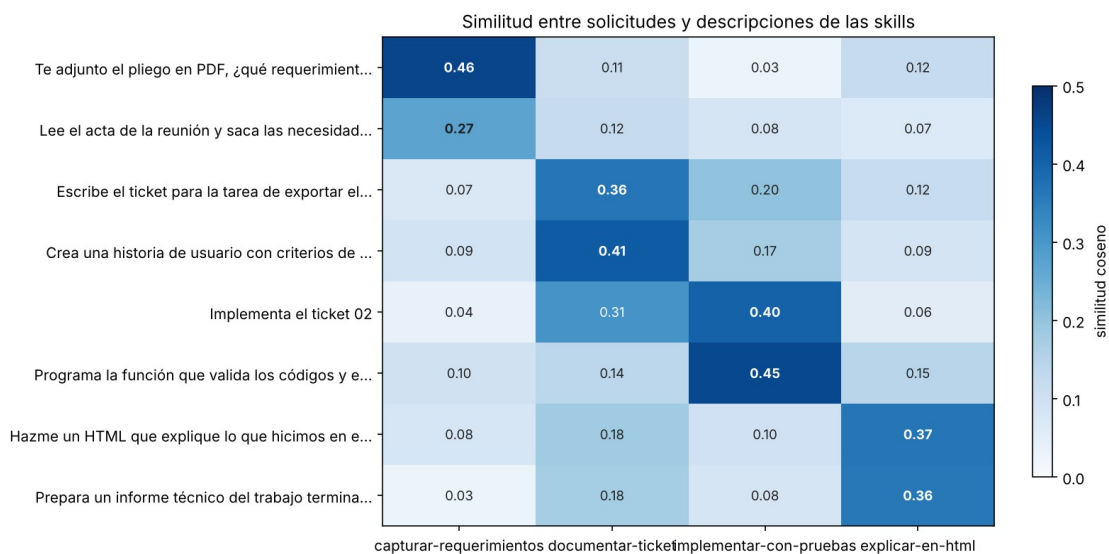


Figura 2. Similitud entre ocho solicitudes y las descripciones de las cuatro skills; en negrita, la elegida.

Las ocho solicitudes se asignan a la skill correcta. El caso más estrecho es «Implementa el ticket 02» (0.40 frente a 0.31, margen de 0.097): la palabra *ticket* aparece en la descripción de *documentar - ticket* y en la de *implementar - con - pruebas*, y la decide el verbo. Es lo esperable, porque las dos skills trabajan sobre el mismo objeto y se distinguen por la acción; por eso cada descripción empieza con el verbo de lo que hace. En los demás casos el margen está entre 0.15 y 0.34. La prueba es solo un piso: un agente real razona sobre la descripción y entiende sinónimos que TF-IDF no capta, pero si las descripciones ya se separan sin conocimiento del idioma, es poco probable que un modelo de lenguaje las confunda.

2.10 Demostración: el flujo completo sobre los documentos de la clase

La skill del numeral 4 debe capturar requerimientos de *documentos provistos*. Los documentos provistos en el curso son los siete capítulos que entrega el profesor (144 páginas en total), y se usaron como entrada de la demostración. Son una prueba exigente: la mayor parte del texto es teoría llena de frases con «se debe» que no piden nada; lo que se pide está en las secciones de ejercicios, escrito en imperativo («Tome una tabla de datos...», «Simule el comportamiento...»);

hay reglas de selección («Desarrolle tres ejercicios de los 6 propuestos»); y las condiciones de entrega del curso están al final del capítulo 1, después de la bibliografía.

El flujo fue el de la sección 2.3: se capturaron los requerimientos de los siete documentos, se cruzaron con lo entregado en el repositorio del equipo, los ejercicios pendientes del capítulo 7 se documentaron como tickets, se implementaron los dos primeros tickets del numeral 4 (agente de mantenimiento) y el resultado se explicó en HTML.

Captura de requerimientos: detección. El script se ejecutó una vez por documento y produjo 183 candidatos. Para medir su calidad se escribió a mano, leyendo los PDF, una referencia independiente con los 45 elementos que piden los documentos: 34 ejercicios, los 4 sub-ítems del numeral 4 del capítulo 7, 3 reglas de selección y los 4 ítems de la lista «Con respecto al desarrollo del curso» del capítulo 1.

Tabla 8. Candidatos por nivel de confianza y su precisión frente a la referencia.

Confianza	Criterio	Candidatos	Son requerimientos	Precisión
Alta	Dentro de una sección de ejercicios	41	41	1.00
Media	Ítem de una lista introducida por «...:»	24	4	0.17
Baja	Prosa con «debe», «se requiere», «podría»	118	0	0.00
Total		183	45	

Los 45 elementos de la referencia aparecen entre los candidatos de confianza alta o media, así que leer esos 65 candidatos basta para no perder nada. La confianza alta es exacta; la media trae, junto a las cuatro condiciones de entrega, 20 ítems de listas de pasos de la teoría (los pasos de la programación genética y de un proyecto de ML) que el agente descarta. Para valorar las reglas nuevas se repitió la medición solo con la regla de obligación, que era la de la primera versión del detector: habría encontrado **5 de los 45** elementos, porque los ejercicios casi nunca dicen «debe».

Captura de requerimientos: revisión del agente. Sobre los candidatos de confianza alta y media, el agente descartó el ítem «Crear un grupo de WhatsApp» (logística, no afecta entregables); renumeró el capítulo 2, que tiene dos ejercicios con el número 3; separó la condición «En alguno de los siguientes problemas, usar datos de alguna entidad gubernamental», que la extracción dejó dentro del ejercicio 6.1, y la sugerencia «Se sugiere usar Ollama», pegada al final del numeral 7.2; partió el numeral 7.4 en su cabecera y sus cuatro funciones; marcó como *Elegibles* los ejercicios de los capítulos con regla de selección; y convirtió lo ambiguo en preguntas abiertas.

Tabla 9. Requerimientos capturados de los siete documentos (requerimientos.md).

Grupo	Cantidad	Ejemplos
Condiciones de entrega (RES-01 a RES-04)	4	Entregar en GitHub; programas en Jupyter o Colab; cada programa en un documento; documentar el uso de IA generativa
Ejercicios obligatorios (RF, Debe)	24	RF-1.1 a 1.5, RF-2.1 a 2.4, RF-5.1 a 5.5, RF-6.1, RF-7.1 a 7.5 con RF-7.4a a 7.4d
Ejercicios elegibles (RF, Elegible)	14	RF-3.1 a 3.6, RF-4.1 a 4.4, RF-6.2 a 6.5
Reglas y condiciones por capítulo (RES)	5	RES-3.S, RES-4.S, RES-6.S, RES-6.D (datos oficiales), RES-7.O (Ollama, Debería)

Grupo	Cantidad	Ejemplos
Supuestos	3	Sin regla de selección, todo es obligatorio; RES-01 a 04 aplican a todos los capítulos; formatos del agente fijados por el equipo
Preguntas abiertas	5	Dos ejercicios numerados 3 (cap. 2); «hacer 3 o 4» (cap. 4); formato de código, orden e historial (cap. 7)

Estado frente al repositorio. Con los requerimientos identificados, el agente cruzó cada capítulo del repositorio con sus ejercicios, su regla de selección y las condiciones de entrega:

Tabla 10. Cumplimiento de los requerimientos en el repositorio (consultado el 2026-10-04).

Capítulo	Entregado	Regla	RES-02 (Jupyter/Colab)	RES-03 (documento)
1. Introducción	RF-1.1 a 1.5	Sí	No aplica	Sí
2. Autómatas celulares	RF-2.2 a 2.4	Depende de P-02	Sí	Sí
3. Algoritmos genéticos	RF-3.1, 3.3, 3.6	Sí (3 de 6)	Sí	Sí
4. Programación genética	RF-4.1, 4.3	Sí	No: scripts .py	No: solo README
5. Redes neuronales	RF-5.1 a 5.5	Sí	Sí	Sí
6. Machine Learning	RF-6.1 a 6.4 (6.4 con datos oficiales)	Sí	No: scripts .py	Sí
7. IA generativa	RF-7.1	Pendientes 7.2 a 7.5	Sí	Sí

El cruce dejó dos hallazgos que el equipo puede corregir (los capítulos 4 y 6 no cumplen RES-02 y el 4 tampoco RES-03) y la lista de lo pendiente del capítulo 7, que pasó a la siguiente skill.

Documentación de tickets. Lo pendiente del capítulo 7 se cortó en cuatro rebanadas verticales. Como el curso no define el formato del código de repuesto, de la orden ni del historial (pregunta P-04), los tickets 01 y 02 trabajan con el supuesto S-03 del equipo (REP-0000, OT-0000 con estado «abierta», historial en CSV) y lo declaran.

Tabla 11. Tickets generados.

Ticket	Entrega	Requerimientos	Bloqueado por	Criterios	Validación
01	Consultar un repuesto y sus existencias	RF-7.4a, RF-7.4b	Ninguno	5	Válido
02	Registrar una orden de trabajo y actualizar el historial	RF-7.4c, RF-7.4d	Ninguno	5	Válido
03	Conversar con el agente usando las funciones como herramientas	RF-7.4, RES-7.O	01 y 02	3	Válido
04	Chatbot sobre los documentos del curso, con citas	RF-7.3, RES-7.O	Ninguno	4	Válido

Para comprobar el validador se le pasó además un ticket escrito sin plantilla («Arreglar lo del almacén», estimado en 13 puntos, con el criterio «que la búsqueda sea rápida y funcione correctamente»). El validador reportó 14 errores y 1 aviso: siete secciones y cuatro campos faltantes, la estimación fuera de la serie, la falta de *fuera de alcance*, el criterio sin formato *Dado/Cuando/Entonces* y el uso de términos no verificables.

Implementación con pruebas. Para los tickets 01 y 02 se creó la interfaz acordada con `NotImplementedError`, se escribieron las pruebas a partir de los criterios y se ejecutó cada una con `ciclo.py`, primero en rojo y luego en verde. En una sesión real el agente alterna rojo y verde criterio por criterio; en el notebook los ciclos se agruparon por ticket para no repetir el módulo completo, pero cada prueba pasó por las dos fases y la bitácora lo registra. Las parejas criterio → prueba salen del script de trazabilidad.

Tabla 12. Resultado del ciclo rojo-verde por criterio (bitacora_tdd.json).

Criterio	Prueba	Rojo	Verde
CA-01.1	test_busqueda_devuelve_descripcion_y_equipos	falla	pasa
CA-01.2	test_rechaza_codigo_con_formato_invalido (4 casos)	falla	pasa
CA-01.3	test_codigo_valido_no_registrado_devuelve_none	falla	pasa
CA-01.4	test_existencias_de_repuesto_en_almacen	falla	pasa
CA-01.5	test_existencias_de_repuesto_ausente_es_cero	falla	pasa
CA-02.1	test_primera_orden_es_ot_0001_y_queda_abierta	falla	pasa
CA-02.2	test_consecutivo_sigue_al_mayor_existente	falla	pasa
CA-02.3	test_falla_vacia_se_rechaza_sin_crear_orden (2 casos)	falla	pasa
CA-02.4	test_historial_inexistente_se_crea_con_encabezado	falla	pasa
CA-02.5	test_segundo_registro_conserva_el_primero	falla	pasa

Las 10 pruebas fallaron en rojo por la razón correcta (`NotImplementedError`) y pasaron en verde: 20 ejecuciones registradas, todas con el resultado esperado. La suite completa tiene 14 casos, porque dos pruebas están parametrizadas, y la trazabilidad confirma 5 de 5 criterios cubiertos en cada ticket. Para comprobar que `ciclo.py` detecta la razón equivocada se ejecutó una prueba que importa un módulo inexistente: la marcó como «falla por la razón equivocada (`ModuleNotFoundError`)».

Algunas decisiones tomadas al implementar, documentadas en el informe HTML: el catálogo y el almacén se reciben como diccionarios porque el curso no entrega datos de repuestos; los códigos en minúsculas se rechazan mientras el formato sea el del supuesto S-03; el consecutivo de las órdenes es el mayor existente más uno, porque contar las órdenes repetiría identificadores si hay huecos (OT-0001, OT-0005); y la fecha de la orden es un parámetro opcional para que las pruebas no dependan del reloj.

Explicación en HTML. El último paso armó `contenido.json` con la evidencia generada (requerimientos, salida de `pytest`, bitácora y código de los módulos) y lo convirtió con el script de la skill. El resultado es un archivo de 16 KB con las ocho secciones obligatorias y 0 advertencias; lo único escrito a mano son los textos explicativos.

Agente de mantenimiento: consultas de repuestos y órdenes de trabajo

Numeral 4 del ejercicio 7.11 · tickets 01 y 02 · flujo de skills del Ejercicio 7.11.1

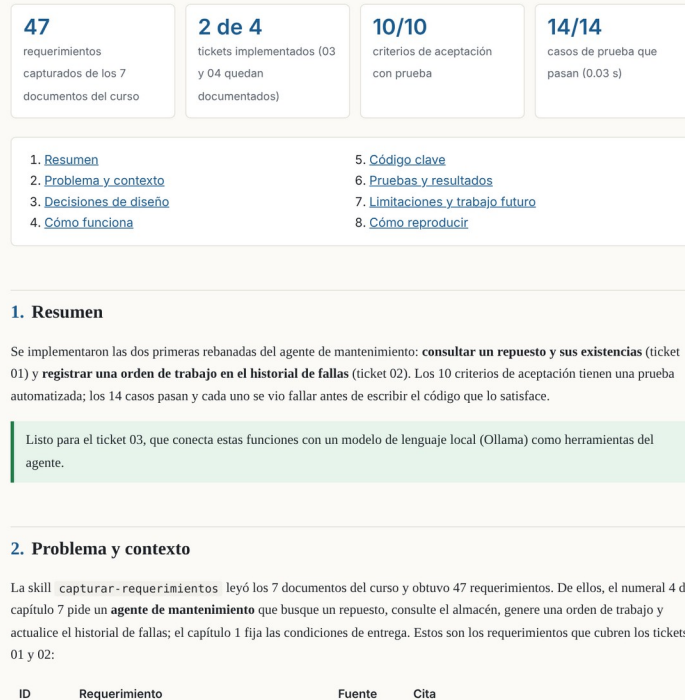


Figura 3. Parte superior del informe HTML generado por la skill `explicar-en-html` (`demo/salida/explicacion.html`).

2.11 Conclusiones

1. Se definieron las cuatro skills que pide el enunciado, cada una como una carpeta con `SKILL.md`, plantillas y scripts, a partir de las de AI Hero: `capturar-requerimientos` (numeral 4), `documentar-ticket` (numeral 1), `implementar-con-pruebas` (numeral 2) y `explicar-en-html` (numeral 3). Las cuatro cumplen el formato.
2. Aplicada a los siete documentos de la clase, la skill de captura encontró los 45 elementos que piden, todos con confianza alta o media, y la confianza alta tuvo precisión de 1.00. Con la sola regla de «debe» habría encontrado 5: en documentos de un curso los requerimientos están en imperativo y dentro de secciones de ejercicios.
3. La decisión de diseño de mayor peso fue separar criterio y mecánica: las instrucciones cubren lo que requiere juicio y seis scripts lo que debe salir igual cada vez. El detector redujo la lectura a 65 candidatos, pero hizo falta el criterio del agente para descartar la logística, separar condiciones pegadas a otros ejercicios y formular 5 preguntas abiertas.
4. Los requerimientos sirvieron para más que los tickets: al cruzarlos con el repositorio aparecieron los ejercicios pendientes del capítulo 7 y dos incumplimientos de las condiciones del curso en los capítulos 4 y 6.
5. Las skills se encadenan por identificadores: una línea de un documento de la clase (capítulo 7, p.23 l.15) se vuelve un requerimiento (RF-7.4c), un criterio (CA-02.1), una prueba y una fila del informe HTML. Los scripts de validación y trazabilidad comprueban esa cadena sin intervención manual.

6. Las descripciones se distinguen entre sí: una similitud léxica simple asigna bien las 8 solicitudes de prueba, con un margen mínimo de 0.097 en el caso que comparte la palabra ticket.
7. La demostración deja implementados los dos primeros tickets del numeral 4 del taller, con 14 casos de prueba que pasan, y documentados el tercero (el agente con un modelo local) y el cuarto (el chatbot del numeral 3 sobre los documentos de la clase).

3. Uso de herramientas de IA Generativa

Los parámetros de entrega del curso establecen que, cuando se utilicen herramientas de IA Generativa, se deben hacer comentarios sobre qué herramientas se emplearon, los prompts utilizados y hasta qué punto sirvieron para el desarrollo de la documentación, los ejercicios o los proyectos. Este capítulo atiende esa exigencia.

3.1 Herramientas empleadas

Tabla 13. Herramientas empleadas en el desarrollo del capítulo.

Herramienta	Uso que se le dio	Utilidad observada
Asistente de IA Claude (Anthropic)	Revisión de las skills de AI Hero, borradores de las cuatro skills y sus scripts, construcción y ejecución del notebook y redacción de este documento.	Alta, siempre que sus afirmaciones se contrastaran con las salidas de los programas (sección 3.3).
Repositorio mattpocock/skills (AI Hero)	Fuente de los archivos SKILL.md de /to-spec, /to-tickets, /implement, /tdd, /teach y /grill-with-docs.	Indispensable: permitió basarse en el texto real de las skills y no en una descripción.
Documentación de Agent Skills (Anthropic)	Reglas del formato: campos name y description, límites y carga progresiva.	Necesaria para el verificador de la sección 2.9.
Documentación de pytest y scikit-learn	Parametrización, tmp_path y códigos de salida de pytest; TfidfVectorizer.	Necesaria para ciclo.py y la prueba de activación.

3.2 Prompts utilizados

Las solicitudes principales hechas a la herramienta fueron del siguiente tipo:

- Desarrollar el primer punto del taller en un notebook de Jupyter, siguiendo la estructura del repositorio del curso.
- Revisar las skills publicadas en AI Hero y proponer cuatro skills propias equivalentes a las del enunciado, en español.
- Tomar como entrada del primer punto los archivos de la clase (los siete capítulos del curso) y verificar los resultados obtenidos.
- Elaborar el documento del capítulo con la misma estructura de los documentos de los capítulos anteriores.

3.3 Alcance y limitaciones observadas

Durante el trabajo ocurrió varias veces que la herramienta produjo resultados incorrectos, que se corrigieron al contrastarlos con las salidas de los programas:

- Un primer informe HTML contaba un caso de prueba menos que pytest: la expresión regular que contaba los casos no reconocía el caso parametrizado cuya falla es solo espacios. Se corrigió leyendo la línea de resumen de pytest.
- La primera versión del detector clasificaba «Consultar en el almacén la cantidad disponible» como requerimiento no funcional, porque la palabra «disponible» activaba la regla de disponibilidad; la regla se restringió a «disponibilidad».

- La primera versión del detector solo reconocía frases con «debe» y viñetas; al aplicarla a los documentos de la clase no veía los ejercicios, escritos en imperativo. Las versiones siguientes también fallaron: los ítems de la teoría que arrastraban un párrafo con «debemos» salían con confianza alta, y la línea «conclusiones.» de un enunciado partido cerraba la sección de ejercicios del capítulo 5, que perdía los ejercicios 4 y 5. Se corrigió exigiendo que los títulos de sección sean la línea completa y empiecen en mayúscula, y midiendo cada cambio contra la referencia escrita a mano.
- Al presentar los resultados de la detección se dijo que había «39 ejercicios»; el conteo correcto, verificado contra la referencia, es 34 ejercicios, 4 sub-ítems y 3 reglas de selección.
- El validador de tickets solo aceptaba criterios que empezaran con «Dado» y rechazó «Dada una pregunta...», que es correcto en español; se corrigió para aceptar dado, dada, dados y dadas.
- En el análisis de activación se escribió inicialmente que el margen mínimo era «al menos 0.10», cuando el valor es 0.097.

Una limitación del ejercicio: las skills se probaron ejecutando sus scripts y siguiendo sus instrucciones paso a paso en el notebook, y la elección automática de la skill se aproximó con TF-IDF. No se midió el comportamiento de un agente real con las skills instaladas, que es el paso natural para continuar.

4. Programas, repositorio y ejecución

4.1 Estructura de los archivos

```
7. IA Generativa/
├── README.md
├── requirements.txt
├── Documento_IA_Generativa.pdf
├── notebooks/
│   └── Ejercicio_1_Definicion_de_Skills.ipynb
├── documentos_clase/      los siete PDF de la clase (entrada de la demostración)
├── skills/                las cuatro skills (las escribe el notebook)
│   ├── capturar-requerimientos/
│   ├── documentar-ticket/
│   ├── implementar-con-pruebas/
│   └── explicar-en-html/
├── demo/                  salida del flujo completo (sección 10 del notebook)
│   ├── texto/ candidatos/ salida/ proyecto/
│   ├── dist/              cada skill en .zip
│   └── img/                figuras generadas por el notebook
```

4.2 Ejecución en Google COLAB

El notebook se puede abrir directamente en Colab con el siguiente enlace y ejecutarse con *Entorno de ejecución* → *Ejecutar todas*; no requiere GPU. pandas, scikit-learn y pytest vienen preinstalados y pypdf se instala automáticamente; las carpetas `skills/` y `demo/` se crean en el directorio de trabajo. Si la carpeta `documentos_clase/` no está, el notebook pide subir los siete PDF de la clase.

- [Ejercicio 1. Definición de skills propias](#)

4.3 Ejecución local con Jupyter

```
pip install -r requirements.txt
jupyter notebook notebooks/
```

El notebook tarda unos 15 segundos en CPU. El resultado no depende de ejecuciones anteriores: la carpeta `demo/` se borra al empezar la demostración. La única cifra que varía entre ejecuciones es el tiempo de la suite de pruebas.

4.4 Instalación de las skills en un agente

Tabla 14. Instalación de las skills según el agente.

Agente	Dónde se copian	Cómo se usan
Claude Code	~/.claude/skills/ (todas las sesiones) o .claude/skills/ dentro del proyecto	Automáticamente por la descripción, o a mano con /capturar-requerimientos, /documentar-ticket, etc.
Claude (web o escritorio)	Configuración → Capacidades → Skills, subiendo el .zip de demo/dist/	Automáticamente por la descripción
Otros agentes con el formato Agent Skills	Carpeta de skills del agente (por ejemplo .agents/skills/)	Según el agente

5. Bibliografía

- [1] Martínez, J. J. (2026). Capítulos 1 a 7 del curso Inteligencia Artificial y Minirobots: Introducción a la IA, Autómatas celulares, Algoritmos genéticos, Programación genética, Redes neuronales, Machine Learning e Inteligencia Artificial Generativa. Universidad Nacional de Colombia.
- [2] Pocock, M. (2026). AI Hero: Skills. <https://www.aihero.dev/> y repositorio <https://github.com/mattpocock/skills> (licencia MIT). Consultado en octubre de 2026.
- [3] Anthropic (2025). Agent Skills. <https://docs.claude.com/en/docs/agents-and-tools/agent-skills/overview>
- [4] Especificación abierta Agent Skills. <https://agentskills.io>
- [5] Beck, K. (2002). Test-Driven Development: By Example. Addison-Wesley.
- [6] Cohn, M. (2004). User Stories Applied: For Agile Software Development. Addison-Wesley.
- [7] Salton, G. y Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513-523.